

# Stopwatch

클라이언트 사이드 로직 조작을 통한 서버 측 검증 우회 (Client-Side Logic Tampering to Server-Side Validation Bypass)

|                          |                                 |               |
|--------------------------|---------------------------------|---------------|
| TARGET<br>problem01      | PREPARED FOR<br>8low8lowme Team | SEVERITY: LOW |
| AUTHOR<br>정슬기 (설계) / 구현팀 | CWE<br>CWE-602 · CWE-807        | STATUS: FINAL |

## 1. 개요 (Executive Summary)

problem01(Stopwatch, 스톱워치 게임)을 분석한 결과, 정답 판정 로직 전체가 브라우저에서 실행되는 자바스크립트(stopwatch.js) 안에 존재하며, 관련 변수와 함수가 전역(window) 스코프에 그대로 노출되어 있음을 확인했다. 개발자 도구만으로 목표 시간 값을 임의로 바꾸거나, 스톱워치를 실제로 조작하지 않고도 성공 처리 함수를 직접 호출할 수 있었다.

서버(api/flag.php)는 클라이언트가 전송한 elapsed\_time 값이 실제 플레이 과정에서 만들어진 값인지 검증할 방법이 없이, 숫자 조건만 맞으면 그대로 플래그를 반환하도록 되어 있었다. 결과적으로 클라이언트 로직을 신뢰한 설계 하나가 게임 자체를 무의미하게 만드는 인증/검증 우회로 이어졌다.

## 2. 공격 시나리오 (Attack Narrative)

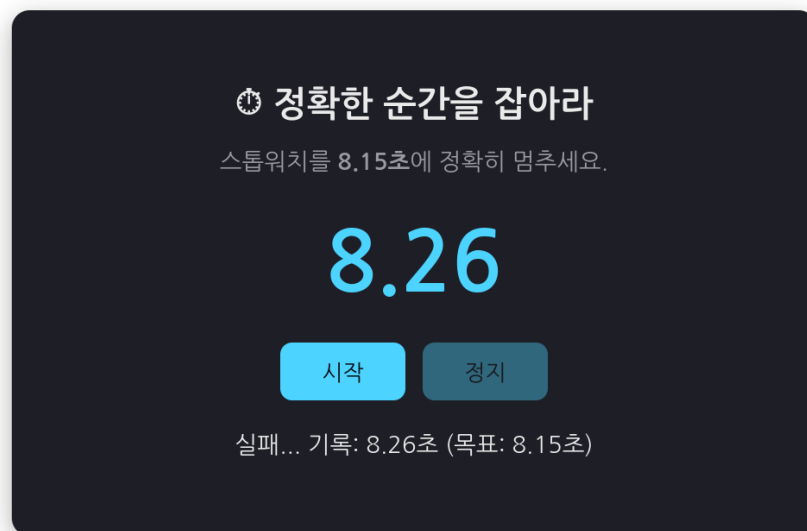
| 단계 | 기법                                                 | 확보한 정보 / 결과                                                              |
|----|----------------------------------------------------|--------------------------------------------------------------------------|
| 1  | 웹 페이지 접속 후 마우스로 8.15초 맞추기 시도                       | 반응 속도의 물리적 한계로 실패 메시지 반복 출력                                              |
| 2  | 개발자 도구(F12) Sources 탭에서 stopwatch.js 소스 분석         | gameConfig.targetTime, checkWinner(), sendFlagRequest()가 전역에 노출되어 있음을 확인 |
| 3  | 콘솔에서 gameConfig.targetTime = 0 으로 변조 후 즉시 정지 버튼 클릭 | 정지 즉시 목표치와 일치 판정되어 checkWinner() 통과, sendFlagRequest() 자동 호출             |

|   |                                                   |                                              |
|---|---------------------------------------------------|----------------------------------------------|
| 4 | (대안) 게임 진행 없이 콘솔에서<br>sendFlagRequest(8.15) 직접 호출 | 스톱워치 조작 없이도 서버로 완료 신호<br>전송, 서버가 값만 보고 성공 처리 |
| 5 | 서버 응답(JSON) 확인                                    | success: true와 함께 최종 FLAG 반환                 |

### 3. 상세 재현 절차 (Proof of Concept)

#### ■ 3.1 정찰 — 스톱워치 UI 및 정상 흐름 확인

problem01 페이지(index.php)에 접속하면 시작/정지 버튼과 함께 스톱워치가 하나 있다. 목표는 정확히 8.15초에 정지하는 것이며, 사람의 반응 속도로는 오차 범위( $\pm 0.05$ 초) 안에 들어오기가 사실상 불가능하다는 점을 먼저 확인했다.



#### ■ 3.2 소스 코드 분석 — 전역 노출 지점 확인

개발자 도구의 Sources 탭에서 stopwatch.js를 열람했다. 목표 시간(8.15초)과 허용 오차 값이 담긴 gameConfig 객체가 let 선언으로 그대로 노출되어 있어, 콘솔에서 직접 읽고 수정할 수 있는 상태임을 확인했다. (checkWinner(), sendFlagRequest() 등 판정 함수 역시 같은 파일 내에 전역으로 노출되어 있었다.)

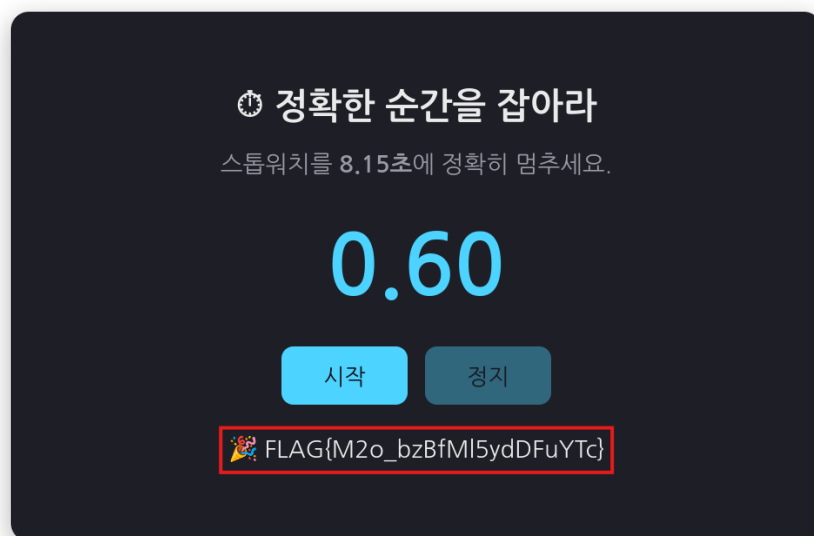
```
let gameConfig = {
  targetTime: 8.15,
  tolerance: 0.05
```

### 3.3 성공 함수 직접 호출을 통한 검증 자체 우회

스톱워치를 아예 실행하지 않고도, 콘솔에서 서버로 완료 신호를 보내는 함수를 목표 시간 값과 함께 직접 호출하는 것만으로 우회할 수 있었다. 해당 함수가 전역에 노출되어 있어 별도의 권한 검증 없이 누구나 호출 가능했다.

```
sendFlagRequest(8.15);
```

### 3.4 FLAG 획득



## 4. 근본 원인 분석 (Root Cause)

- 성공 판정 로직(checkWinner)이 서버가 아닌 클라이언트 자바스크립트 안에 존재해, 브라우저에서 완전히 열람·수정·실행이 가능함
- gameConfig, checkWinner(), sendFlagRequest() 등 핵심 변수/함수가 별도의 은닉 없이 전역(window) 스코프에 노출되어 있어 콘솔에서 직접 접근 가능함
- 서버(api/flag.php)는 클라이언트가 보낸 elapsed\_time 값이 실제 플레이로 만들어진 값인지 검증할 수단이 없이, 값의 범위만 확인하고 그대로 신뢰함
- 클라이언트 측 로직은 사용자 경험을 위한 보조 장치일 뿐이며, 최종 성공/실패 판단은 반드시 서버가 독립적으로 수행해야 한다는 원칙이 지켜지지 않음